

LAB DAY 3

NAME: AGILAN VIYASH

REG. NO.: 192511346

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

NUMBER OF WAYS TO MOVE BALL OUT OF GRID

PYTHON CODE:

```
def find_paths(m, n, N, i, j):  
    # dp[r][c] = number of ways to reach cell (r, c)  
    dp = [[0] * n for _ in range(m)]  
    dp[i][j] = 1  
  
    result = 0  
  
    for _ in range(N):  
        new_dp = [[0] * n for _ in range(m)]  
  
        for r in range(m):  
            for c in range(n):  
                if dp[r][c] == 0:  
                    continue  
  
                # Move up  
                if r == 0:  
                    result += dp[r][c]  
                else:  
                    new_dp[r - 1][c] += dp[r][c]
```

```
# Move down
if r == m - 1:
    result += dp[r][c]
else:
    new_dp[r + 1][c] += dp[r][c]
```

```
# Move left
if c == 0:
    result += dp[r][c]
else:
    new_dp[r][c - 1] += dp[r][c]
```

```
# Move right
if c == n - 1:
    result += dp[r][c]
else:
    new_dp[r][c + 1] += dp[r][c]
```

```
dp = new_dp
```

```
return result
```

```
# Sample Input
```

```
m = 2
```

```
n = 2
```

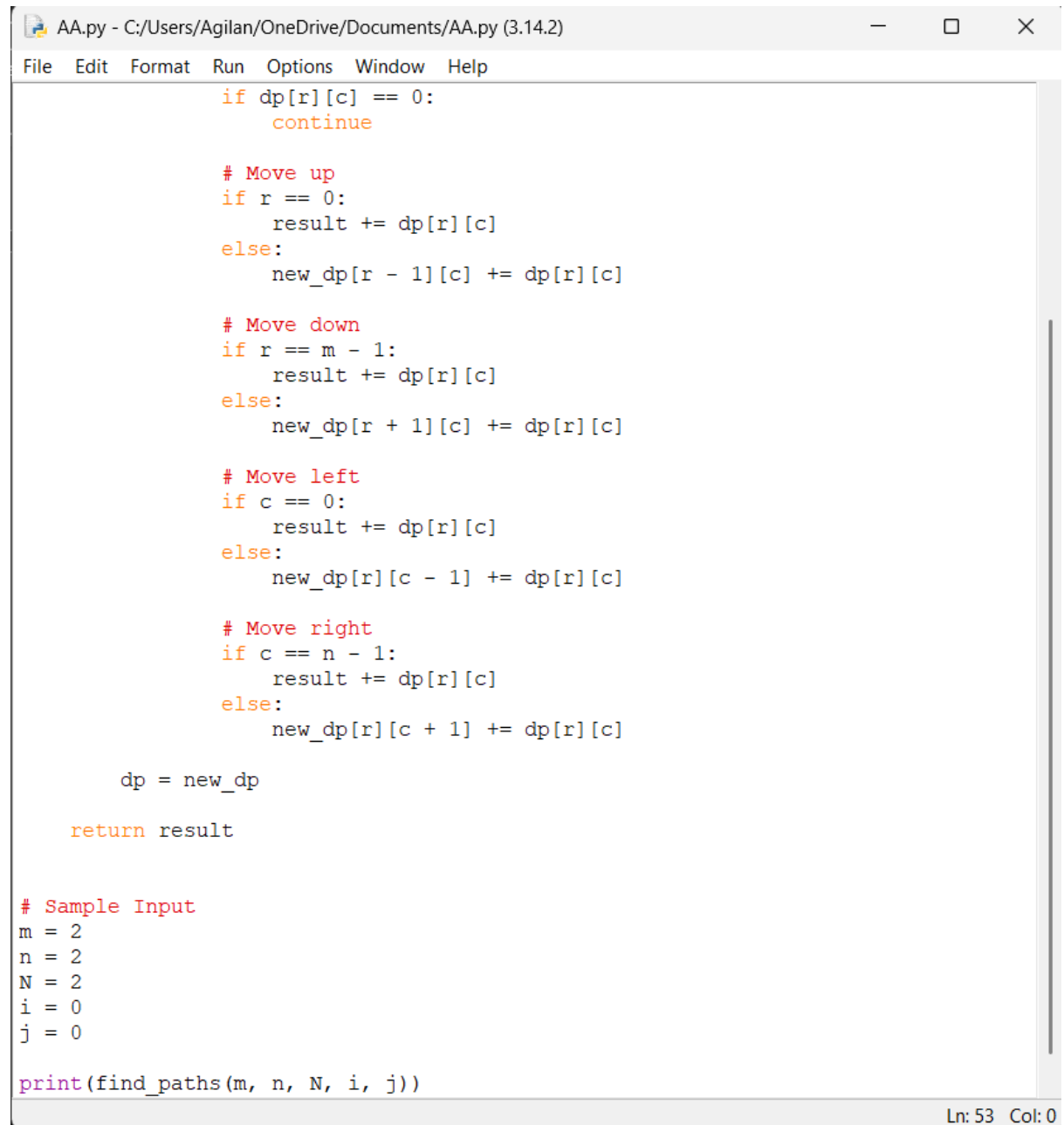
```
N = 2
```

```
i = 0
```

```
j = 0
```

```
print(find_paths(m, n, N, i, j))
```

EXECUTION:



The screenshot shows a Python IDE window titled "AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code is as follows:

```
        if dp[r][c] == 0:
            continue

        # Move up
        if r == 0:
            result += dp[r][c]
        else:
            new_dp[r - 1][c] += dp[r][c]

        # Move down
        if r == m - 1:
            result += dp[r][c]
        else:
            new_dp[r + 1][c] += dp[r][c]

        # Move left
        if c == 0:
            result += dp[r][c]
        else:
            new_dp[r][c - 1] += dp[r][c]

        # Move right
        if c == n - 1:
            result += dp[r][c]
        else:
            new_dp[r][c + 1] += dp[r][c]

    dp = new_dp

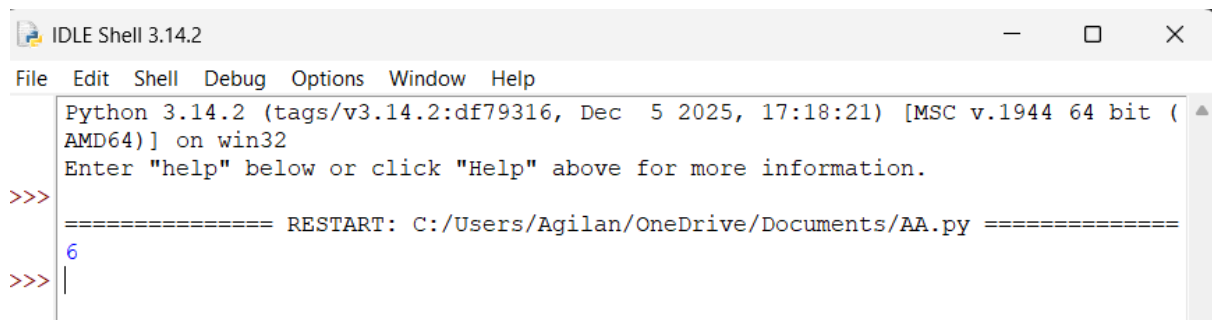
    return result

# Sample Input
m = 2
n = 2
N = 2
i = 0
j = 0

print(find_paths(m, n, N, i, j))
```

The status bar at the bottom right indicates "Ln: 53 Col: 0".

OUTPUT:



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
6
>>> |
```

HOUSE ROBBER II

PYTHON CODE:

```
def rob_linear(nums):
```

```
    prev = 0
```

```
    curr = 0
```

```
    for money in nums:
```

```
        new_curr = max(curr, prev + money)
```

```
        prev = curr
```

```
        curr = new_curr
```

```
    return curr
```

```
def rob_houses(nums):
```

```
    n = len(nums)
```

```
    if n == 0:
```

```
        return 0
```

```
    if n == 1:
```

```
        return nums[0]
```

```
# Case 1: Exclude the first house
```

```
case1 = rob_linear(nums[1:])
```

```
# Case 2: Exclude the last house
```

```
case2 = rob_linear(nums[:-1])
```

```
return max(case1, case2)
```

```
# Sample Input
```

```
nums = [2, 3, 2]
```

```
print(rob_houses(nums))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def rob_linear(nums):
    prev = 0
    curr = 0

    for money in nums:
        new_curr = max(curr, prev + money)
        prev = curr
        curr = new_curr

    return curr

def rob_houses(nums):
    n = len(nums)

    if n == 0:
        return 0

    if n == 1:
        return nums[0]

    # Case 1: Exclude the first house
    case1 = rob_linear(nums[1:])

    # Case 2: Exclude the last house
    case2 = rob_linear(nums[:-1])

    return max(case1, case2)

# Sample Input
nums = [2, 3, 2]

print(rob_houses(nums))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
3
>>>
```

CLIMBING STAIRS

PYTHON CODE:

```
def climb_stairs(n):

    if n <= 2:
```

```
    return n
```

```
    first = 1
```

```
    second = 2
```

```
    for _ in range(3, n + 1):
```

```
        third = first + second
```

```
        first = second
```

```
        second = third
```

```
    return second
```

```
# Sample Input
```

```
n = 4
```

```
print(climb_stairs(n))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def climb_stairs(n):
    if n <= 2:
        return n

    first = 1
    second = 2

    for _ in range(3, n + 1):
        third = first + second
        first = second
        second = third

    return second

# Sample Input
n = 4

print(climb_stairs(n))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
5
>>> |
```

UNIQUE PATH IN A GRID

PYTHON CODE:

```
def unique_paths(m, n):
```

```
    dp = [[1] * n for _ in range(m)]
```

```
    for i in range(1, m):
```

```
        for j in range(1, n):
```

```
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
```

```
    return dp[m - 1][n - 1]
```

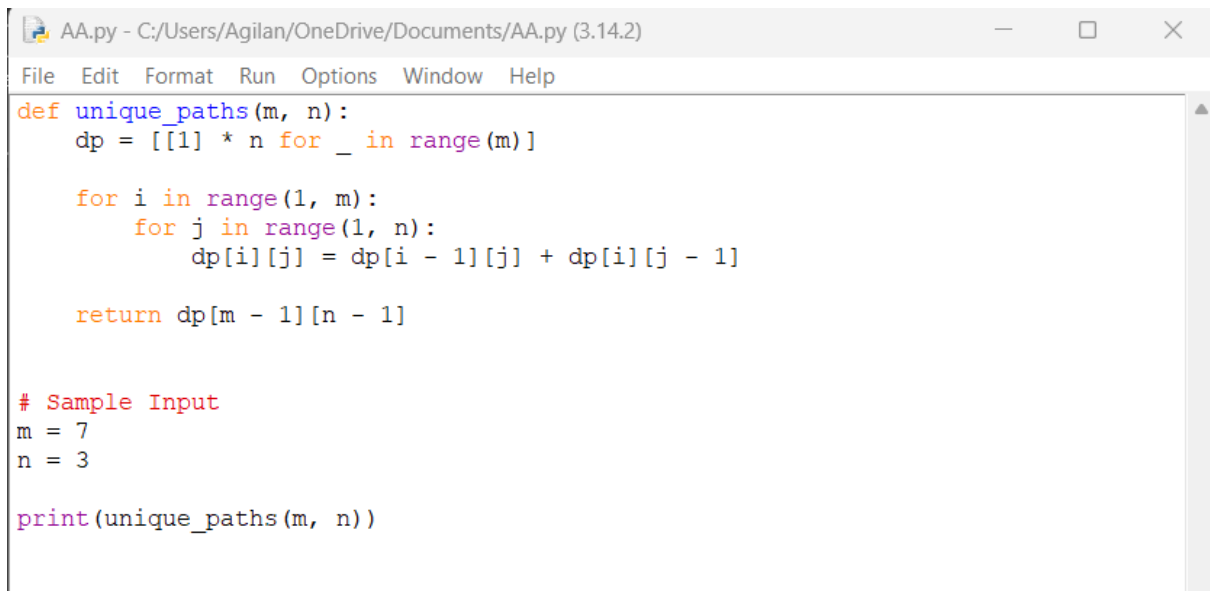

Sample Input

m = 7

n = 3

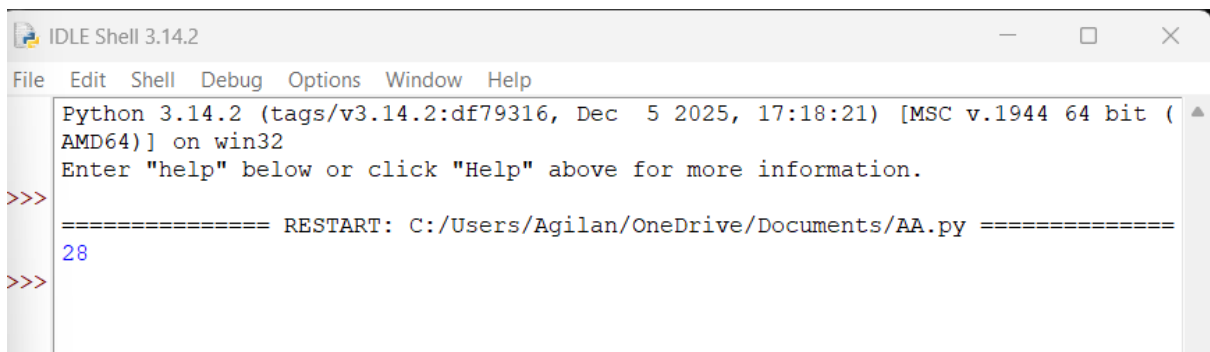
print(unique_paths(m, n))

EXECUTION:

A screenshot of a Python IDE window titled "AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor contains the following Python code:

```
def unique_paths(m, n):  
    dp = [[1] * n for _ in range(m)]  
  
    for i in range(1, m):  
        for j in range(1, n):  
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]  
  
    return dp[m - 1][n - 1]  
  
# Sample Input  
m = 7  
n = 3  
  
print(unique_paths(m, n))
```

OUTPUT:

A screenshot of an IDLE Shell window titled "IDLE Shell 3.14.2". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The shell displays the following text:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====  
28  
>>>
```

LARGES GROUPS IN A STRING

PYTHON CODE:

def large_groups(s):

```
result = []
start = 0

for i in range(1, len(s) + 1):

    # End of a group
    if i == len(s) or s[i] != s[i - 1]:

        # Check group size
        if i - start >= 3:
            result.append([start, i - 1])

        start = i

return result
```

```
# Sample Input
s = "abbxxxzzy"
```

```
print(large_groups(s))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def large_groups(s):
    result = []
    start = 0

    for i in range(1, len(s) + 1):

        # End of a group
        if i == len(s) or s[i] != s[i - 1]:

            # Check group size
            if i - start >= 3:
                result.append([start, i - 1])

            start = i

    return result

# Sample Input
s = "abbxxxxzzy"

print(large_groups(s))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
[[3, 6]]
>>>
```

GAME OF LIFE

PYTHON CODE:

```
def game_of_life(board):
```

```
    rows = len(board)
```

```
    cols = len(board[0])
```

```
    directions = [
```

```
        (-1, -1), (-1, 0), (-1, 1),
```

```
(0, -1),      (0, 1),  
(1, -1), (1, 0), (1, 1)  
]
```

```
new_board = [row[:] for row in board]
```

```
for r in range(rows):
```

```
    for c in range(cols):
```

```
        live_neighbors = 0
```

```
        for dr, dc in directions:
```

```
            nr = r + dr
```

```
            nc = c + dc
```

```
            if 0 <= nr < rows and 0 <= nc < cols:
```

```
                live_neighbors += board[nr][nc]
```

```
        # Live cell
```

```
        if board[r][c] == 1:
```

```
            if live_neighbors < 2 or live_neighbors > 3:
```

```
                new_board[r][c] = 0
```

```
            else:
```

```
                new_board[r][c] = 1
```

```
        # Dead cell
```

```
        else:
```

```
            if live_neighbors == 3:
```

```
                new_board[r][c] = 1
```

```
return new_board
```

```
# Sample Input
```

```
board = [  
    [0, 1, 0],  
    [0, 0, 1],  
    [1, 1, 1],  
    [0, 0, 0]  
]
```

```
result = game_of_life(board)
```

```
print(result)
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
new_board = [row[:] for row in board]

for r in range(rows):
    for c in range(cols):

        live_neighbors = 0

        for dr, dc in directions:
            nr = r + dr
            nc = c + dc

            if 0 <= nr < rows and 0 <= nc < cols:
                live_neighbors += board[nr][nc]

        # Live cell
        if board[r][c] == 1:
            if live_neighbors < 2 or live_neighbors > 3:
                new_board[r][c] = 0
            else:
                new_board[r][c] = 1

        # Dead cell
        else:
            if live_neighbors == 3:
                new_board[r][c] = 1

    return new_board

# Sample Input
board = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 1, 1],
    [0, 0, 0]
]

result = game_of_life(board)

print(result)
```

Ln: 50 Col: 13

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
>>>
```

CHAMPAGNE TOWER

PYTHON CODE:

```
def champagne_tower(poured, query_row, query_glass):
```

```
    # Create a tower with 100 rows
```

```
    glasses = [[0.0] * (i + 1) for i in range(100)]
```

```
    glasses[0][0] = poured
```

```
    for i in range(99):
```

```
        for j in range(i + 1):
```

```
            if glasses[i][j] > 1:
```

```
                overflow = glasses[i][j] - 1
```

```
                glasses[i + 1][j] += overflow / 2
```

```
                glasses[i + 1][j + 1] += overflow / 2
```

```
            glasses[i][j] = 1
```

```
    return min(1, glasses[query_row][query_glass])
```

```
# Sample Input
```

```
poured = 2
```

```
query_row = 1
```

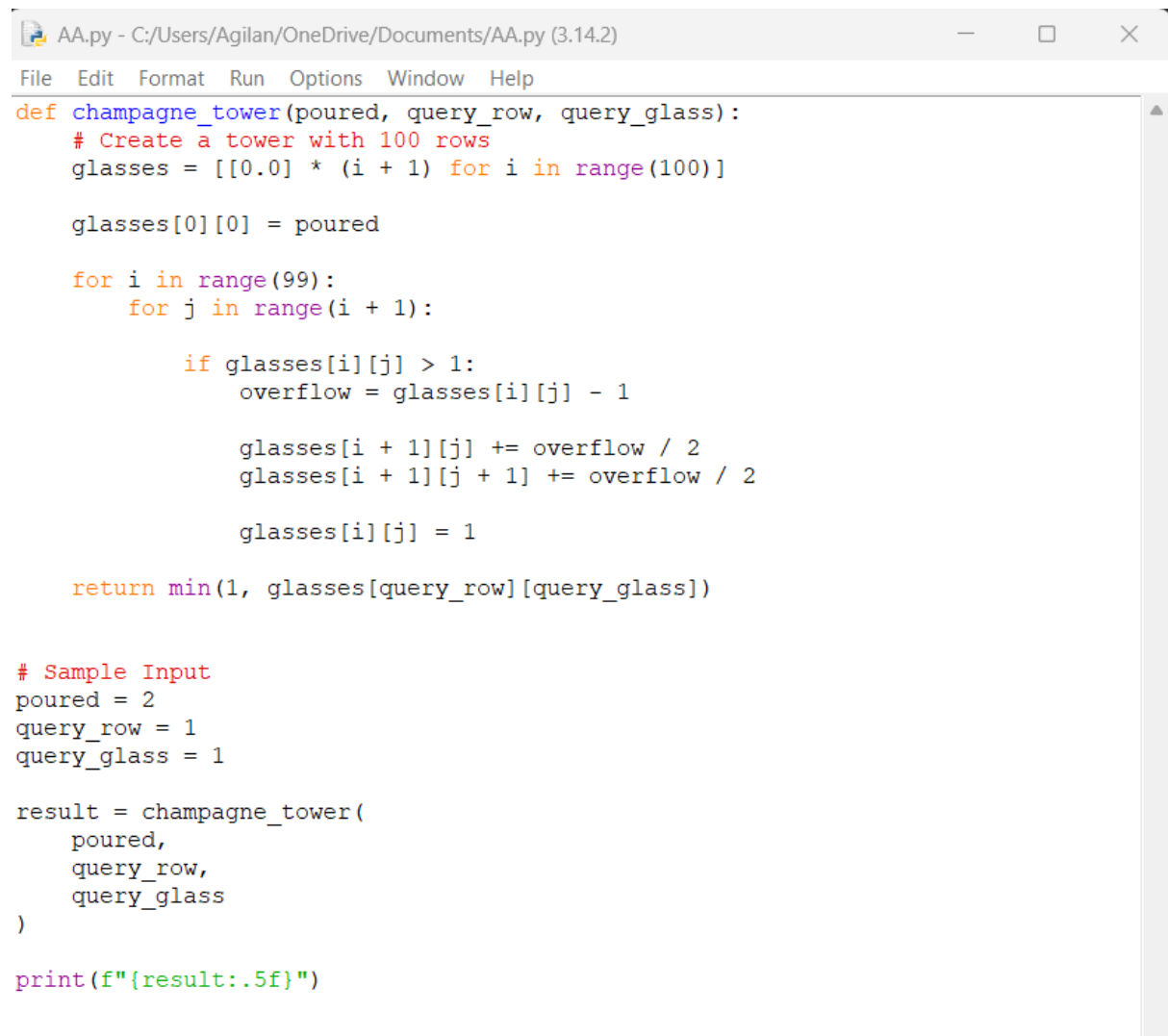
```
query_glass = 1
```

```
result = champagne_tower(
```

```
    poured,
```

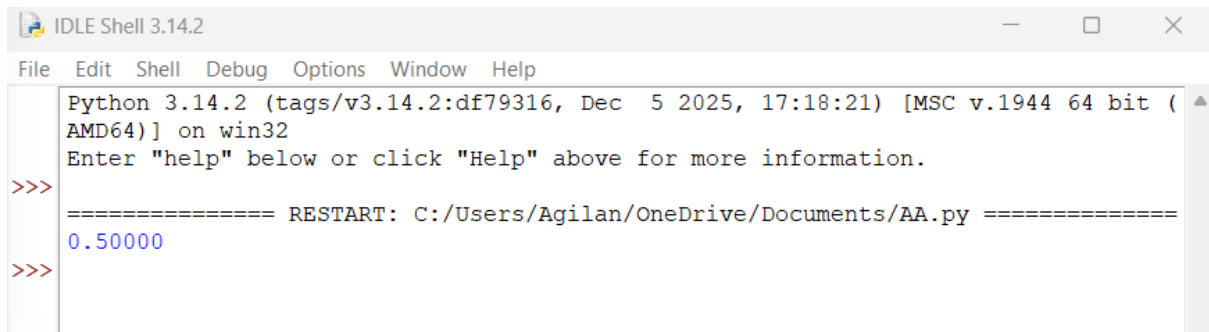
```
    query_row,  
    query_glass  
)  
  
print(f'{result:.5f}')
```

EXECUTION:

A screenshot of a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The window contains a Python script for a champagne tower simulation. The script defines a function 'champagne_tower' that takes 'poured', 'query_row', and 'query_glass' as arguments. It creates a 100x100 grid of glasses, initializes the first glass with the 'poured' value, and simulates the flow of champagne through the tower. The script then uses sample input values (poured=2, query_row=1, query_glass=1) to calculate the result and prints it with 5 decimal places.

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)  
File Edit Format Run Options Window Help  
def champagne_tower(poured, query_row, query_glass):  
    # Create a tower with 100 rows  
    glasses = [[0.0] * (i + 1) for i in range(100)]  
  
    glasses[0][0] = poured  
  
    for i in range(99):  
        for j in range(i + 1):  
            if glasses[i][j] > 1:  
                overflow = glasses[i][j] - 1  
  
                glasses[i + 1][j] += overflow / 2  
                glasses[i + 1][j + 1] += overflow / 2  
  
                glasses[i][j] = 1  
  
    return min(1, glasses[query_row][query_glass])  
  
# Sample Input  
poured = 2  
query_row = 1  
query_glass = 1  
  
result = champagne_tower(  
    poured,  
    query_row,  
    query_glass  
)  
  
print(f'{result:.5f}')
```

OUTPUT:



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
0.50000
>>>
```